

CLDP (Custom Lightweight Discovery Protocol) Specification

Dadi Sasank Kumar (22CS10020)
CS39006 Networks Laboratory
Assignment 7

March 31, 2025

Contents

1	Introduction	2
2	Protocol Overview	2
2.1	Message Types	2
2.2	CLDP Header Format	2
2.3	CLDP Payload Format	3
3	IP Packet Crafting and Parsing	3
3.1	Raw Socket Usage	3
3.2	Packet Assembly and Parsing	3
4	Server Implementation (<code>cldp_server.c</code>)	4
5	Client Implementation (<code>cldp_client.c</code>)	5
6	Demonstration Scenario	5
7	Build and Run Instructions	6
7.1	Compiling the Code	6
7.2	Running the Server and Client	6
7.3	Capturing Traffic	6
8	Assumptions and Limitations	6

1 Introduction

This document specifies the Custom Lightweight Discovery Protocol (CLDP), designed and implemented for Assignment 7 of the CS39006 Networks Laboratory course. CLDP uses raw sockets in C (POSIX standard) for node discovery and metadata exchange within a closed network environment. Nodes can announce their presence and query others for application-level metadata such as CPU load, system time, or hostname. The protocol operates directly over IPv4.

2 Protocol Overview

CLDP bypasses standard transport layers (TCP/UDP) and functions directly at the IP layer. It utilizes raw sockets (`AF_INET`, `SOCK_RAW`) with the custom IP protocol number 253. All aspects of the IP header, including the checksum, are manually constructed and managed by the application.

2.1 Message Types

CLDP defines three primary message types, identified by the first byte of the custom header:

- **HELLO (0x01):** Periodically broadcast by server nodes (every 10 seconds in the current implementation) to announce their active presence on the network.
- **QUERY (0x02):** Sent by a client node, typically via broadcast, to request specific metadata from other nodes. The implementation supports querying for:
 - 1: Hostname
 - 2: System Time (Unix timestamp with microseconds)
 - 3: CPU Load (1-minute average, obtained via ‘uptime’)
- **RESPONSE (0x03):** Sent by a server node directly back to the originator of a QUERY message. It contains the requested metadata or an error indication.

2.2 CLDP Header Format

Every CLDP message begins with a mandatory 8-byte header immediately following the IP header. The fields are defined as follows:

Offset (Bytes)	0	1	2-3	4-7
Field	Msg Type	Payload Len	Transaction ID	Reserved
Size (Bytes)	1	1	2	4

- **Message Type (1 byte):** Indicates the message purpose (0x01: HELLO, 0x02: QUERY, 0x03: RESPONSE).
- **Payload Length (1 byte):** Specifies the length, in bytes, of the CLDP payload data that follows this header. Does *not* include the size of the CLDP header itself. Maximum payload size is 255 bytes.

- **Transaction ID (2 bytes):** A unique identifier used to match a RESPONSE message with its corresponding QUERY message. Sent in Network Byte Order (`htons/ntohs`). HELLO messages may contain an arbitrary Transaction ID.
- **Reserved (4 bytes):** Reserved for future protocol extensions. Must be set to 0 by senders and ignored by receivers in this version.

2.3 CLDP Payload Format

The format of the data following the CLDP header depends on the Message Type:

- **HELLO:** Contains a fixed payload string (e.g., "HELLO\0") indicating the node is active. The 'Payload Len' field reflects the length of this string, including the null terminator.
- **QUERY:** Contains a single byte indicating the type of metadata being requested (1 for Hostname, 2 for System Time, 3 for CPU Load). The 'Payload Len' field is 1.
- **RESPONSE:** Contains the requested metadata formatted as a null-terminated C string (e.g., "myhostname\0", "date.time.seconds\0", "0.15\0", or an error message like "Error: Unknown Query Type (4)\0"). The 'Payload Len' field reflects the length of this string, including the null terminator.

3 IP Packet Crafting and Parsing

3.1 Raw Socket Usage

The implementation relies exclusively on raw sockets:

- **IP Checksum:** A standard checksum algorithm (RFC 1071) is implemented and used to calculate the checksum for the crafted IP header before sending. The checksum field in the header must be zero during calculation.

3.2 Packet Assembly and Parsing

- **Assembly (Sending):**
 1. A buffer is prepared.
 2. The IP header fields (Version, IHL, TOS, Total Length, ID, Flags/Frag Offset, TTL, Protocol=253, Source IP, Destination IP) are filled. Total Length includes IP header, CLDP header, and CLDP payload. Source IP is determined dynamically using `getifaddrs`. Checksum is initially 0.
 3. The IP header checksum is calculated over the IP header bytes and inserted into the header.
 4. The 8-byte CLDP header is constructed immediately following the IP header in the buffer. Fields (Msg Type, Payload Len, Transaction ID) are set appropriately (using `htons` for Trans ID).

5. The CLDP payload (if any) is copied into the buffer immediately after the CLDP header.
6. The entire buffer (up to the IP Total Length) is sent using `sendto()`.

- **Parsing (Receiving):**

1. A packet is received using `recvfrom()`.
2. The IP header's Protocol field is checked. If not 253, the packet is ignored.
3. The IP header length (IHL field * 4) is determined.
4. (Optional but recommended) The received IP header checksum is verified.
5. The CLDP header is located immediately after the IP header.
6. The CLDP Message Type, Payload Length, and Transaction ID (`ntohs`) are extracted.
7. The CLDP payload is located immediately after the CLDP header, and its length is validated against the received packet size and the CLDP Payload Length field.
8. The message is processed based on its Type and Payload content.

4 Server Implementation (`cldp_server.c`)

The CLDP server performs two main tasks concurrently:

- **Announcing Presence:** Periodically (every 10 seconds), it constructs and broadcasts a CLDP HELLO packet (Msg Type 0x01) to the network broadcast address (255.255.255.255).
- **Responding to Queries:** It continuously listens for incoming packets on the raw socket.
 - It uses `select()` with a short timeout to avoid blocking indefinitely, allowing the periodic HELLO task to run.
 - Upon receiving a packet, it parses it as described above.
 - If the packet is a valid CLDP QUERY (Msg Type 0x02), it extracts the requested meta-data type from the payload.
 - It retrieves the requested data (hostname via `gethostname()`, time via `gettimeofday()`, load via parsing `uptime` output).
 - It constructs a CLDP RESPONSE packet (Msg Type 0x03) containing the retrieved data (or an error string) as the payload. The Transaction ID from the QUERY is copied into the RESPONSE header.
 - The RESPONSE packet is sent unicast directly back to the source IP address of the QUERY packet.

The server includes helper functions for IP checksum calculation and dynamically finding a usable local IP address.

5 Client Implementation (`cldp_client.c`)

The CLDP client allows a user to query the network for specific metadata:

- Presents the user with a choice of metadata to query (Hostname, System Time, CPU Load).
- Constructs a CLDP QUERY packet (Msg Type 0x02) based on the user's selection. The payload is a single byte representing the chosen type. A random Transaction ID is generated for the query.
- Sends the QUERY packet to the network broadcast address (255.255.255.255).
- Enters a listening phase to wait for responses.
 - It uses `poll()` with a timeout to wait for incoming packets.
 - It parses received packets.
 - If a packet is a valid CLDP RESPONSE (Msg Type 0x03) *and* its Transaction ID matches the ID of the QUERY sent, the client considers it a valid reply.
 - The source IP address and the payload (the requested metadata string) from valid responses are displayed to the user.
 - The client continues listening for potential responses from multiple servers until the timeout expires.
- After the timeout, the client prompts the user for another query or to exit.

6 Demonstration Scenario

A typical demonstration involves the following steps:

1. Start one or more instances of the `cldp_server` on different machines on the same LAN (or on the same machine for testing, being mindful of loopback behavior). Each server will begin broadcasting HELLO packets.
2. Start the `cldp_client` on one machine.
3. The client prompts the user to select a metadata type (1, 2, or 3).
4. The client broadcasts a QUERY packet for the selected metadata type.
5. Each active server receives the QUERY, retrieves the requested information, and sends a unicast RESPONSE packet back to the client.
6. The client receives the RESPONSE packets (matching the Transaction ID) and displays the information along with the source IP of the responding server.
7. Network traffic can be captured using tools like `tcpdump` or `Wireshark` to observe the HELLO, QUERY, and RESPONSE packets (filtering by IP protocol number 253).

7 Build and Run Instructions

7.1 Compiling the Code

```
gcc -o client cldp_client.c
gcc -o server cldp_server.c
```

7.2 Running the Server and Client

Executing these programs requires root privileges due to the use of raw sockets.

- To start the server:

```
sudo ./server
```

- To start the client:

```
sudo ./client
```

Run the server(s) first, then run the client to initiate queries.

7.3 Capturing Traffic

Used `tcpdump` to monitor CLDP traffic. Filter by the custom IP protocol number(here, 253).

- **To see packets on the any interface.** This is useful for seeing broadcasts (HELLO, QUERY) leaving or entering the machine:

```
sudo tcpdump -i any -X -n proto 253
```

8 Assumptions and Limitations

- **Network Environment:** Designed for closed LAN environments.
- **IPv4 Only:** The implementation is strictly for IPv4.
- **Privileges:** Requires root privileges (`sudo`) to create and use raw sockets.
- **Error Handling:** Basic error checking for system calls is present, but robustness against malformed packets or complex network errors is limited. Client input validation only checks for integer input but not the range (1-3 explicitly within the processing logic).
- **Payload Size Limit:** CLDP payload is limited to 255 bytes due to the 1-byte Payload Length field.
- **Loopback Behavior:** When client and server run on the *same machine*:

- Broadcasts (HELLO, QUERY) sent by the local machine might be seen on both the loopback ('lo') and physical interfaces(OS optimization).
- Unicast responses (RESPONSE) sent from the local server to the local client are often routed *only* over the loopback interface ('lo') by the operating system, bypassing the physical network hardware.